

Translated

From: e-maxx.ru

Fenwick Tree

Let f be some group operation (a binary associative function over a set with an identity element and inverse elements) and A be an array of integers of length N . Denote f 's infix notation as $*$; that is, $f(x, y) = x * y$ for arbitrary integers x, y . (Since this is associative, we will omit parentheses for order of application of f when using infix notation.)

The Fenwick tree is a data structure which:

- calculates the value of function f in the given range $[l, r]$ (i.e. $A_l * A_{l+1} * \dots * A_r$) in $O(\log N)$ time
- updates the value of an element of A in $O(\log N)$ time
- requires $O(N)$ memory (the same amount required for A)
- is easy to use and code, especially in the case of multidimensional arrays

The most common application of a Fenwick tree is *calculating the sum of a range*. For example, using addition over the set of integers as the group operation, i.e. $f(x, y) = x + y$: the binary operation, $*$, is $+$ in this case, so $A_l * A_{l+1} * \dots * A_r = A_l + A_{l+1} + \dots + A_r$.

The Fenwick tree is also called a **Binary Indexed Tree** (BIT). It was first described in a paper titled "A new data structure for cumulative frequency tables" (Peter M. Fenwick, 1994).

Description

Overview

For the sake of simplicity, we will assume that function f is defined as $f(x, y) = x + y$ over the integers.

Suppose we are given an array of integers, $A[0 \dots N-1]$. (Note that we are using zero-based indexing.) A Fenwick tree is just an array, $T[0 \dots N-1]$, where each element is equal to the sum of elements of A in some range, $[g(i), i]$:

$$T_i = \sum_{j = g(i)}^i A_j$$